

PROJECT SOUNDING LINE

The Software Verification and Parallel Test Infrastructure System

PART II

Execution Infrastructure and Parallel Runtime

Version 1.1 Amendment Edition

Amendment purpose

Eliminate human-mediated validation-lane diplomacy by formalizing queued lease requests, automatic fair handoff, stale-source guards, multi-lane browser capacity, environment-pressure routing, and scheduler behavior that prevents shared validation resources from turning continuous integration into a traffic jam.

Effective August 10, 2026 | Document ID CS-SL-P2-001 v1.1

A. Document Control and Amendment Authority

Field	Value
Program	Project Sounding Line
Subsystem	The Software Verification and Parallel Test Infrastructure System
Document	Governing Architecture and Product Requirements Document - Part II, Version 1.1 Amendment Edition
Base document	Version 1.0, July 28, 2026
Version	1.1
Effective date	August 10, 2026
Document ID	CS-SL-P2-001 v1.1
Status	Governing amendment; additive and superseding only where clauses are explicitly named
Amendment trigger	Project Helm Phase 1 closure and repeated cross-project validation/integration churn exposed gaps between intended evidence reuse and operational behavior.

Normative incorporation rule

The Version 1.0 body remains in force in full except where this Version 1.1 amendment explicitly adds, clarifies, or supersedes a clause. This amendment must never be read as permission to weaken required proof. Its purpose is to preserve correctness while preventing redundant proof, avoidable serialization, and closure loops that do not increase confidence.

This amendment is deliberately narrow. It does not change the three-document authority split: Part I defines verification meaning, Part II defines execution machinery, and Part III defines repository/Codex/release enforcement.

1. Amendment Summary and Supersession Map

Version 1.0 already defines a scheduler, fairness age, resource-specific leases, dynamic throttling, port leasing, browser sharding, and durable ownership. Version 1.1 makes those mechanisms mandatory for cross-project queueing and removes manual chat coordination as an accepted steady-state mechanism.

v1.0 area	v1.1 amendment
4 Scheduler Behavior	Adds cross-project queue classes, aging, source guards, and automatic handoff.
5 Runtime Lease Broker	Adds queued lease request records and deterministic handoff.
6 Resource Catalog	Adds validation-lane and reference-environment resource classes.
16-18 Browser / Port / Sharding	Adds multi-lane port pools and isolated browser lane bundles.
25 Timeout Architecture	Separates product timeout from environment-pressure timeout.
30-31 Capacity / Throttling	Adds interactive-load classification and deferral/rerouting.
32 Local Execution	Adds contaminated-environment evidence rules.
33 CI/Distributed	Makes clean hosted runner a reference environment for timing-sensitive proof.
39 Acceptance Matrix	Adds seven-project queue and starvation-free handoff scenarios.

2. Validation Queue Domain

Term	Definition
Validation Queue	Broker-owned ordered set of pending execution requests for scarce resources.
Lease Request	Immutable request for a resource bundle tied to source, plan, gate, priority, and owner.
Queue Position	Observable order plus age/priority explanation; not a promise of wall-clock start time.
Fairness Age	Monotonic priority boost that prevents starvation.
Source Guard	Required fetch/identity check immediately before scarce-resource acquisition.
Automatic Handoff	Broker transition from released owner to next eligible request without manual coordination.
Validation Lane	Isolated bundle of ports, server, database clone, browser slot, storage, artifacts, and build identity.
Reference Environment	Known capacity/software profile suitable for timing-sensitive authoritative proof.

```
REQUESTED -> QUEUED -> ELIGIBLE -> ACQUIRING -> ACTIVE -> RELEASING -> RELEASED
      |           |
      |           +--> CANCELLED_STALE (source guard failed)
      +--> CANCELLED / EXPIRED / SUPERSEDED
```

3. Queue Ordering and Fairness

Queue order is broker policy, not cross-chat negotiation. Humans and Codex may request priority classes, but they may not manually seize or delete another run’s lease.

Signal	Effect
Already ACTIVE	Never preempt ordinary active work. Let governed timeout/cancel/cleanup finish.
POST_MERGE_INTEGRATED_SHA	High decision-unblocking priority for short exact-main proof.
RELEASE_CRITICAL / MAINLINE_CANDIDATE	Elevated priority over ordinary focused diagnostics.
Fairness age	Pending request gains priority to prevent starvation.
Expected duration	May help schedule short sentinel between long runs when it does not starve older work.
Resource compatibility	Requests needing different isolated lanes may run concurrently.
Source staleness	Stale request is cancelled/replanned before scarce resource acquisition.

No more validation diplomacy

Messages such as “Tideglass goes next, then Wakebook, then Helm” may exist as human context, but they are not the scheduling mechanism. The broker must hold the queue, reveal ownership, and hand the lane to the next eligible request automatically.

4. Source Guard Before Expensive Acquisition

A candidate waiting thirty minutes for a browser lane may become stale before it starts. The broker must therefore revalidate source and gate assumptions immediately before expensive resource acquisition.

Fetch or verify current canonical main according to the gate contract.

Confirm the candidate source SHA/tree is unchanged.

Evaluate whether mainline movement materially invalidates the planned proof.

If stale, transition request to CANCELLED_STALE without opening ports, copying databases, starting servers, or launching browsers.

Replan/reconcile outside the scarce lane, then enqueue a new request.

Resource rule

A stale candidate must fail before it consumes the thing everyone is waiting for.

5. Multi-Lane Browser Execution

A single compatibility port may remain, but the target architecture is a pool of independently leased browser lanes. Each lane receives a complete isolated bundle.

Lane resource	Isolation requirement
Application port	Run-owned dynamic lease; 3100 is a compatibility preference, not a universal mutex.
API/helper ports	Run-owned, identity-checked.
Database	Run-owned SQLite clone or MySQL namespace.
Browser worker/context	Run/node identity, version, memory weight, project.
Storage/media root	Run-owned synthetic root.
Build output	Source-bound task-owned output/cache identity.
Evidence root	Run-owned immutable result path after completion.

When capacity allows two fully isolated lanes, unrelated browser suites may run simultaneously. When workstation capacity does not allow it, the scheduler serializes them explicitly and reports queue wait separately from test duration.

6. Automatic Handoff and Lease Lifecycle

ACTIVE OWNER

- > TEARING_DOWN
- > CLEANUP_VERIFIED
- > RELEASED
- > broker selects next ELIGIBLE request
- > source guard
- > atomic bundle acquire
- > ACTIVE next owner

A released port without a released lease is not a handoff.

A stale lock filename without a live/owned lease is not ownership; broker state is authoritative.

Cleanup must finish before the next incompatible owner becomes ACTIVE.

Queued callers should receive status updates without polling process lists or coordinating through chat.

No caller may delete, overwrite, or bypass the queue because the previous run appears slow.

7. Interactive Load and Environment Pressure

Local execution must preserve interactive reserve. When a developer is running a heavy game, render, compile, or other resource-intensive workload, the scheduler should detect resource pressure rather than allowing cold-start/browser timing noise to masquerade as application failure.

Pressure class	Scheduler behavior	Evidence meaning
NORMAL	Run according to ordinary capacity profile.	Timing + functional evidence eligible.
ELEVATED	Reduce concurrency; postpone low-priority expensive nodes.	Functional evidence valid; timing claims qualified.
CONTENTED	Defer timing-sensitive browser/build nodes or route to hosted/reference worker.	Do not classify local timing failure as product defect without reference reproduction.
CRITICAL	Do not start new expensive nodes; preserve active runs if safe.	Environment/infrastructure status only.

BF6 is not a test framework

A workstation compiling Next.js while a GPU-heavy game is running may be a useful torture test, but it is not the default reference environment. Sounding Line should qualify the environment, not teach every feature to survive arbitrary local compile latency by widening waits forever.

8. Timeout Architecture Amendment

Timeouts must separate product no-progress limits, test-case budgets, setup/cold-start budgets, queue budgets, and outer controller budgets. One number must not represent all five.

Timeout	Meaning
Queue timeout	How long a request may wait before it is reclassified/escalated; does not mean product failed.
Setup/cold-start budget	Dependency install, generation, server compile/warmup; environment/infrastructure evidence.
Test action timeout	Bounded wait for a user-visible or API condition after setup is ready.
Test total budget	Maximum complete suite/journey duration under declared environment.
Controller hard timeout	Safety ceiling for the executor; triggers governed cancellation/cleanup.

9. Cross-Project Queue Visibility

Every pending and active scarce-resource request should be observable through one machine-readable queue projection. It must reveal enough to coordinate without exposing secrets.

Field	Purpose
requestId / project / phase	Who is waiting.
gate / suite / planDigest	What proof is requested.
sourceSha / baseSha	Which exact source will run.
priorityClass / fairnessAge	Why it is ordered where it is.
requiredBundle	Which scarce resources are blocking execution.
state / queuedAt / eligibleAt	Lifecycle timing.
estimatedDurationClass	Scheduling hint, never evidence.
blockReason	Resource, dependency, source-stale, capacity, or policy.

10. Part II v1.1 Acceptance Additions

Seven concurrent projects can enqueue browser/resource work without chat-mediated ordering.

Fairness aging prevents indefinite starvation.

Active owner is never preempted by ordinary later request.

Automatic cleanup-to-next-owner handoff works without deleting lock files.

Stale source cancels before expensive resource allocation.

Two isolated browser lanes may execute concurrently when capacity profile permits.

Local interactive pressure can defer/reroute timing-sensitive work without weakening assertions.

Queue wait, setup time, execution time, teardown time, and finalization time are measured separately.

Port 3100 is no longer the architecture's single human-negotiated choke point.

Final Part II v1.1 rule

Scarce resources are scheduled by the broker, not by conversational diplomacy. Queue fairly, revalidate before acquisition, isolate aggressively, hand off automatically, and qualify environment pressure instead of turning contention into feature churn.

R. References and Amendment Closure

Base authority: Project Sounding Line Part II - Execution Infrastructure and Parallel Runtime, Version 1.0, July 28, 2026.
Companion authority: Parts I and III Version 1.1 Amendment Editions. Operational motivation includes the shared validation-lane contention observed during concurrent Helm, Tideglass, Wakebook, Admiralty, Shipwright, Drydock, and Deepwater work.

All unaffected Version 1.0 clauses remain governing. Where Version 1.0 permits implementation latitude that would still require manual queue ownership or one broad global lock, Version 1.1 establishes the target broker behavior above.